FAKE NEWS DETECTION USING COUNTVECTORIZER

1. Introduction

Fake news refers to false or misleading information presented as news. It can spread through websites and social media, influencing public opinion and creating confusion.

This project uses Natural Language Processing (NLP) and machine learning algorithms to classify news articles as **FAKE** or **REAL**. CountVectorizer converts text into numerical features that machine learning models can process.

2. Objectives

- To preprocess news titles and article text.
- To convert text data into numerical features using CountVectorizer.
- To train and compare multiple machine learning models.
- To evaluate model performance using accuracy, precision, recall, and F1-score.
- To identify the best-performing model for the dataset.

3. Dataset Description

The project uses the `news.csv` dataset.

The dataset contains the following columns:

- `Unnamed: 0`: Index column.
- `title`: News headline.
- `text`: News article content.
- `label`: News category, either FAKE or REAL.

Dataset details:

- Total records: 6,335
- FAKE news: 3,164
- REAL news: 3,171
- Training records: 5,068
- Testing records: 1,267

The dataset is divided into 80% training data and 20% testing data.

4. What Is CountVectorizer?

CountVectorizer is a text feature extraction technique provided by the scikit-learn library. It converts text documents into numerical vectors by counting how many times words appear in the documents.

For example:

Documents:

- "AI detects fake news"
- "AI detects real news"

The vocabulary contains the unique words found in the documents. Each document is represented by the count of each vocabulary word.

CountVectorizer helps machine learning algorithms understand text data numerically.

5. Text Preprocessing

The following preprocessing steps are used:

1. Remove records containing missing titles, text, or labels.
2. Combine the news title and article content into a single text column.
3. Convert labels to uppercase for consistency.
4. Split the dataset into training and testing sets.
5. Convert the text into numerical features using CountVectorizer.

6. CountVectorizer Configuration

The project uses the following configuration:

- `lowercase=True`: Converts text to lowercase.
- `stop_words="english"`: Removes common English stop words.
- `max_features=10000`: Limits the vocabulary to 10,000 features.

The resulting training matrix has 5,068 rows and 10,000 features. The testing matrix has 1,267 rows and 10,000 features.

7. Machine Learning Models

The following models are compared:

1. Linear Regression
2. Multinomial Naive Bayes
3. Bernoulli Naive Bayes
4. Logistic Regression
5. Decision Tree
6. Random Forest
7. K-Nearest Neighbors (KNN)
8. Linear Support Vector Machine (Linear SVM)

Each model is trained using the CountVectorizer features and evaluated using the test dataset.

8. Evaluation Metrics

Accuracy: Measures the proportion of correct predictions among all predictions.

Precision: Measures how many news articles predicted as FAKE are actually labelled FAKE.

Recall: Measures how many actual FAKE-labelled articles are correctly identified.

F1-score: Combines precision and recall into a single metric.

These metrics help compare the performance of the different models.

9. Model Comparison Results

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	92.58%	91.40%	94.00%	92.68%
Random Forest	90.69%	90.81%	90.52%	90.66%
Linear SVM	90.13%	89.44%	91.00%	90.21%
Multinomial Naive Bayes	86.98%	86.79%	87.20%	87.00%
KNN	82.40%	80.60%	85.31%	82.89%
Bernoulli Naive Bayes	81.06%	76.31%	90.05%	82.61%
Decision Tree	78.93%	78.77%	79.15%	78.96%

Linear Regression	69.22%	67.58%	73.78%	70.54%

Note: Values are reported from the project notebook. Minor rounding differences may occur.

10. Best-Performing Model

Logistic Regression achieved the best overall results in this experiment.

- Accuracy: 92.58%
- Precision: 91.40%
- Recall: 94.00%
- F1-score: 92.68%

The results indicate that Logistic Regression performed better than the other evaluated models on this dataset and test split.

11. Result Analysis

The comparison shows that Logistic Regression achieved the highest accuracy and F1-score. Random Forest and Linear SVM also performed well.

Multinomial Naive Bayes provided a useful baseline for text classification. Decision Tree and Linear Regression produced lower scores in this experiment.

The results demonstrate that CountVectorizer can be combined with machine learning algorithms to perform text classification.

12. Limitations

- Model performance depends on the quality and representativeness of the dataset.
- CountVectorizer records word counts but does not directly understand context or meaning.
- News articles with unfamiliar vocabulary or different writing styles may be misclassified.
- The Linear SVM training process produced a convergence warning in the notebook.
- A predicted FAKE or REAL label is not proof that a news claim is true or false. Predictions should be independently verified using reliable sources.

13. Conclusion

This project demonstrates a fake news classification workflow using Natural Language Processing, CountVectorizer, and machine learning algorithms.

Eight models were evaluated using accuracy, precision, recall, and F1-score. Logistic Regression achieved the best performance, with 92.58% accuracy and a 92.68% F1-score.

The project illustrates how text can be converted into numerical features and used to train models for automated news classification. Further improvements could include testing additional datasets, tuning model parameters, and comparing CountVectorizer with TF-IDF.

14. Tools and Technologies

- Python
- Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- Scikit-learn
- Natural Language Processing
- CountVectorizer
- Machine Learning

15. References

- Scikit-learn documentation: CountVectorizer
- Scikit-learn documentation: Logistic Regression
- Scikit-learn documentation: Model evaluation metrics

In [10]:
```
# Import the required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.preprocessing import LabelEncoder

from sklearn.naive_bayes import MultinomialNB, BernoulliNB
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import LinearSVC

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score
)
```

In [11]:
```
# Load your CSV file
df = pd.read_csv("news.csv")

print(df.head())
print(df.shape)
print(df["label"].value_counts())
```

```
   Unnamed: 0                                              title  \
0        8476                       You Can Smell Hillary’s Fear
1       10294  Watch The Exact Moment Paul Ryan Committed Pol...
2        3608        Kerry to go to Paris in gesture of sympathy
3       10142  Bernie supporters on Twitter erupt in anger ag...
4         875   The Battle of New York: Why This Primary Matters

                                                text label
0  Daniel Greenfield, a Shillman Journalism Fello...  FAKE
1  Google Pinterest Digg Linkedin Reddit Stumbleu...  FAKE
2  U.S. Secretary of State John F. Kerry said Mon...  REAL
3  — Kaydee King (@KaydeeKing) November 9, 2016 T...  FAKE
4  It's primary day in New York and front-runners...  REAL
(6335, 4)
label
REAL    3171
FAKE    3164
Name: count, dtype: int64
```

In [12]:
```
# Clean and prepare the dataset
df = df.dropna(subset=["title", "text", "label"]).copy()

df["news"] = (
    df["title"].astype(str) + " " + df["text"].astype(str)
)

X = df["news"]
y = df["label"].str.strip().str.upper()

print("Total records:", len(df))
```

```
print("Classes:", y.unique())
print(y.value_counts())
```

```
Total records: 6335
Classes: <ArrowStringArray>
['FAKE', 'REAL']
Length: 2, dtype: str
label
REAL    3171
FAKE    3164
Name: count, dtype: int64
```

In [13]:
```
# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 5068
Testing samples: 1267
```

In [14]:
```
# Apply CountVectorizer
vectorizer = CountVectorizer(
    lowercase=True,
    stop_words="english",
    max_features=10000
)

X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

print("Training feature shape:", X_train_vec.shape)
print("Testing feature shape:", X_test_vec.shape)
print("Vocabulary size:", len(vectorizer.vocabulary_))
```

```
Training feature shape: (5068, 10000)
Testing feature shape: (1267, 10000)
Vocabulary size: 10000
```

In [15]:
```
# Train and compare all models
models = {
    "Linear Regression": "linear",
    "Multinomial NB": MultinomialNB(),
    "Bernoulli NB": BernoulliNB(),
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(
        n_estimators=100, random_state=42, n_jobs=-1
    ),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "SVM": LinearSVC(random_state=42)
}

encoder = LabelEncoder()
y_train_encoded = encoder.fit_transform(y_train)
y_test_encoded = encoder.transform(y_test)
```

```
results = []
trained_models = {}

for name, model in models.items():

    if name == "Linear Regression":
        model = LinearRegression()
        model.fit(X_train_vec, y_train_encoded)

        scores = model.predict(X_test_vec)
        predictions = (scores >= 0.5).astype(int)
        predictions = encoder.inverse_transform(predictions)

    else:
        model.fit(X_train_vec, y_train)
        predictions = model.predict(X_test_vec)

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, predictions),
        "Precision": precision_score(
            y_test, predictions, pos_label="FAKE", zero_division=0
        ),
        "Recall": recall_score(
            y_test, predictions, pos_label="FAKE", zero_division=0
        ),
        "F1 Score": f1_score(
            y_test, predictions, pos_label="FAKE", zero_division=0
        )
    })

    trained_models[name] = model

comparison_df = pd.DataFrame(results).sort_values(
    "F1 Score", ascending=False
).reset_index(drop=True)

print(comparison_df.round(4).to_string(index=False))
```

```
              Model  Accuracy  Precision  Recall  F1 Score
Logistic Regression    0.9258     0.9140  0.9400    0.9268
      Random Forest    0.9069     0.9081  0.9052    0.9066
                SVM    0.9013     0.8944  0.9100    0.9021
     Multinomial NB    0.8698     0.8679  0.8720    0.8700
                KNN    0.8240     0.8060  0.8531    0.8289
       Bernoulli NB    0.8106     0.7631  0.9005    0.8261
      Decision Tree    0.7893     0.7877  0.7915    0.7896
  Linear Regression    0.6922     0.6758  0.7378    0.7054
```

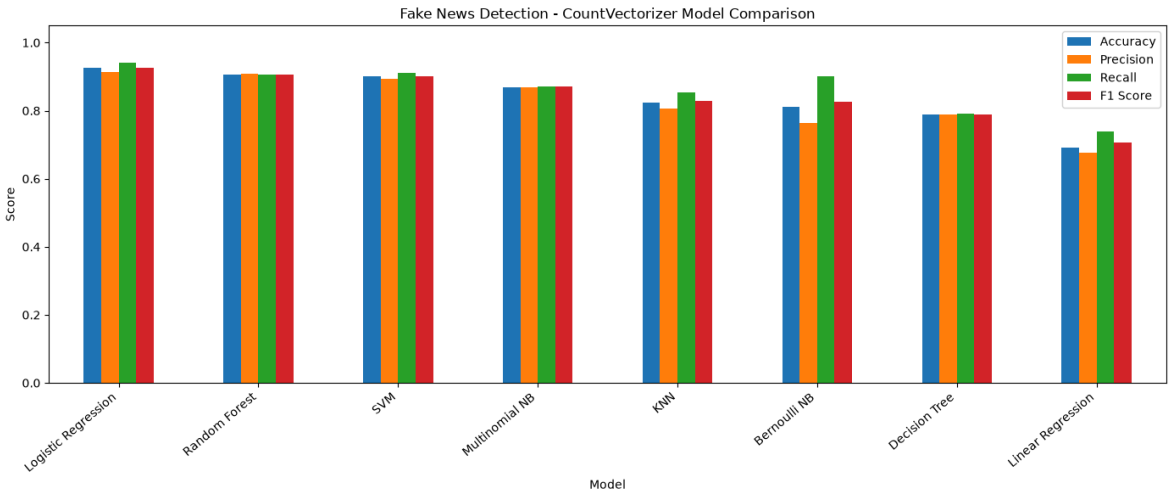
```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\svm\_base.py:1298: Conv
ergenceWarning: Liblinear failed to converge, increase the number of itera
tions.
  warnings.warn(
```

In [17]:

```
# Visualize the comparison
comparison_df.set_index("Model")[
    ["Accuracy", "Precision", "Recall", "F1 Score"]
].plot(kind="bar", figsize=(14, 6))

plt.title("Fake News Detection - CountVectorizer Model Comparison")
```

```
plt.ylabel("Score")
plt.ylim(0, 1.05)
plt.xticks(rotation=40, ha="right")
plt.tight_layout()
plt.show()
```



In [18]:
```
# Identify the best classification model
classification_results = comparison_df[
    comparison_df["Model"] != "Linear Regression"
]

best_model = classification_results.iloc[0]

print("Best model:", best_model["Model"])
print(f"Accuracy: {best_model['Accuracy']:.2%}")
print(f"Precision: {best_model['Precision']:.2%}")
print(f"Recall: {best_model['Recall']:.2%}")
print(f"F1 Score: {best_model['F1 Score']:.2%}")
```

```
Best model: Logistic Regression
Accuracy: 92.58%
Precision: 91.40%
Recall: 94.00%
F1 Score: 92.68%
```

In [19]:
```
# Predict news from new text
news_text = [
    "Government announces a new public health policy after a press confer
]

news_vector = vectorizer.transform(news_text)

prediction = trained_models["Logistic Regression"].predict(news_vector)

print("Predicted label:", prediction[0])
```

```
Predicted label: FAKE
```

In [ ]:

In [ ]:

In [ ]: